MSCPHUB MSCP Disk Server for OpenVMS Nodes

User’s Guide

MSCPHUB-UG-0119

MSCPHUB Version X01-19 *** 22-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

MSCPHUB is free software, distributed under the GNU General Public License, Version 3 or later. It comes with ABSOLUTELY NO WARRANTY.

Contents

Preface **4**
Intended Audience . . . 4
Why MSCPHUB Exists . . . 5
Document Structure . . . 6
Related Documents . . . 7
Trademarks . . . 8
Conventions . . . 9

Chapter 1. Introduction **10**
1.1 What MSCPHUB Does . . . 10
1.2 How an OpenVMS Node Sees the Server . . . 11
1.3 What Is Served . . . 12
1.4 When a WRITE Is Done . . . 13
1.5 What MSCPHUB Does Not Do . . . 14
1.6 Small Computers . . . 15

Chapter 2. Installation **16**
2.1 Requirements . . . 16
2.2 Building and Installing . . . 17
2.3 What Is Installed . . . 18
2.4 Search Paths . . . 19
2.5 Installing in Another Place . . . 20
2.6 Verifying the Installation . . . 21
2.7 Removal . . . 22

Chapter 3. Configuration **23**
3.1 The Facts of the Cluster . . . 23
3.2 Preparing the Disks . . . 24
3.3 The Parameter File . . . 25
3.4 Checking the Configuration . . . 26

Chapter 4. Operation **27**
4.1 Starting the Server . . . 27
4.2 What the Log Shows at Start . . . 28
4.3 Using the Disks on OpenVMS . . . 29
4.4 Displaying the State of the Server . . . 30
4.5 Stopping the Server . . . 31
4.6 Altering the Service . . . 32
4.7 Backing Up the Disks . . . 33
4.8 Monitoring the Server . . . 34
4.9 Recording the Counters in T4 Files . . . 35
4.10 Reading the Counters over Modbus/TCP . . . 36
4.11 Running Without systemd . . . 37

Chapter 5. Command Dictionary **38**
MSCPHUB . . . 39

MHMON 42

Chapter 6. Messages **44**
6.1 Message Format 44
6.2 Message Reference 45

Chapter 7. Troubleshooting **51**
7.1 The Server Stops at Once 52
7.2 No Node Is Seen 53
7.3 No Channel Opens, or It Opens and Closes Again 54
7.4 The Disks Do Not Appear on the Node 55
7.5 The Circuit Closes, or Writes Are Slow 56
7.6 A Command Fails on the Node 57
7.7 The Disks Are Slow 58
7.8 MHMON Shows Nothing 59
7.9 Collecting Information for a Report 60

Appendix A. Files **61**
Environment Variables 62

Appendix B. Limits **63**

Appendix C. Performance **64**
C.1 The Test Configuration 64
C.2 Three Disks Compared 65
C.3 Endurance 66
C.4 Where the Time Goes 67
C.5 The Server on Another Computer 68
C.6 Losing the Server Under Load 69
C.7 Reading These Numbers 70

Appendix D. Modbus Register Map **71**
D.1 Conventions 71
D.2 The Server, from Register 0 72
D.3 A Disk, from Register 1000 + 100 × n 73
D.4 A Node, from Register 2000 + 50 × n 74
D.5 Device Identification 75

Preface

Intended Audience

This manual is for the system manager who installs MSCPHUB on a Linux computer and gives its disks to OpenVMS nodes. It assumes that the reader can work as root on Linux and as SYSTEM on OpenVMS, and that the OpenVMS nodes already form a VMScluster over the LAN. Nothing else is assumed.

The programmer who is to alter the server wants the *MSCPHUB Programmer's Guide* instead. This manual says nothing about how the server is built inside.

Why MSCPHUB Exists

For forty years the VMScluster has shown how computers share their disks: every node sees every disk, a disk is served by whoever has it, and the cluster goes on when a member goes down. The machines of Digital Equipment Corporation are leaving the machine rooms. The systems they ran, and the engineering culture they were made with, should not leave with them.

MSCPHUB is one part of the work to keep that heritage alive and at work. It lets OpenVMS nodes - on Alpha, on Integrity, on x86, or emulated - keep their disks on the storage of today: a file, a partition, an LVM volume of a Linux computer. OpenVMS is not changed in any way; to OpenVMS the server is a disk controller of the cluster, like an HSC.

MSCPHUB is also made the DEC way, on purpose. Its messages have the OpenVMS form and severities, its parameters are qualifiers, it has a HELP library, its versions are IDENTs, its sources follow the conventions of the OpenVMS sources, and this manual is laid out like the manuals of OpenVMS. This is not decoration. A documented decision for everything, a message for every condition, nothing that the operator has to guess - that way of working is part of the heritage, and it is kept by being used.

That is also why MSCPHUB can be read as a course. The student of computer engineering finds in it a real cluster protocol, a disk server and its storage, end to end, small enough to read in a few evenings; the *MSCPHUB Programmer's Guide* is written to be studied. And the engineer who worked with VMS clusters in their day will find the old words in their old places - SCS, MSCP, the virtual circuit, the HSC - and may well remember something the younger ones never knew.

Document Structure

This manual contains the following chapters and appendixes:

Part	Contents
Chapter 1	Introduces the server: what it does, how an OpenVMS node sees it, what it does not do, the small computers it runs on
Chapter 2	Installing the server
Chapter 3	Configuring the server: the facts of the cluster, the disks, the parameter file
Chapter 4	Running the server, using its disks on OpenVMS, watching it with MHMON, T4 and Modbus, running it without systemd
Chapter 5	Contains the command dictionary: the `mscphub` and `mhmon` commands and all their qualifiers
Chapter 6	Describes the format of a message and lists every message with what to do about it
Chapter 7	Troubleshooting: a symptom, what to look at, and what to do
Appendix A	Lists the files installed and used
Appendix B	Lists the limits of the server
Appendix C	Gives the results of measurements of the server
Appendix D	Gives the map of the Modbus registers

The reader who installs the server for the first time wants Chapters 2, 3 and 4, in that order. The reader who holds a message wants Chapter 6, and the one who holds a symptom, Chapter 7.

Related Documents

- *OpenVMS Cluster Systems*
- *Guidelines for OpenVMS Cluster Configurations*
- *OpenVMS System Manager's Manual*
- *MSCPHUB Programmer's Guide*
- The manual page `mscphub(8)` and the help library of the server (`mscphub /HELP`)

Trademarks

OpenVMS, VMScluster, Alpha, AlphaServer, Integrity, Digital, DEC, HSC, HSJ and HSD are trademarks of Hewlett Packard Enterprise Development LP or of VMS Software, Inc. T4 and TLViz are tools of Hewlett Packard Enterprise. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. Intel and Xeon are trademarks of Intel Corporation. Arm and Cortex are trademarks of Arm Limited. Rockchip is a trademark of Fuzhou Rockchip Electronics Co., Ltd. Raspberry Pi is a trademark of Raspberry Pi Ltd. Modbus is a registered trademark of Schneider Electric, licensed to the Modbus Organization, Inc. Debian is a registered trademark of Software in the Public Interest, Inc. DPDK is a project of The Linux Foundation. PF_RING is a trademark of ntop. NaPi is the name of the boards of the NaPi Lab team. All other names of products and companies are trademarks of their respective owners.

MSCPHUB is an independent implementation, made for interoperability. It contains no code of OpenVMS and is not affiliated with, endorsed or supported by any of the owners named here; their names are used only to say what MSCPHUB works with.

Conventions

The following conventions are used in this manual:

Convention	Meaning
`#`	The prompt of the Linux shell of root. Text after the prompt is entered by the user.
`$`	The DCL prompt of an OpenVMS node. Text after the prompt is entered by the user.
italics	A term being defined, or a value to be replaced by one of your own.
`UPPERCASE`	A qualifier, a keyword, an OpenVMS command, or the identification of a message.
`[ ]`	An optional part. Do not type the brackets.
`|`	A choice of one of the items it separates.
`...`	The part before it may be repeated.
`Ctrl/C`	Hold down the Ctrl key and press C.

Qualifiers are shown in uppercase; `mscphub` takes them in any case.

Note

Sample output in this manual is reproduced as the server produces it. The first part of a line of the log - the date, the process and the routine - is left out where it does not matter.

Chapter 1. Introduction

1.1 What MSCPHUB Does

MSCPHUB is an MSCP disk server that runs on Linux. It makes files and block devices of a Linux computer available to the OpenVMS nodes on the same LAN as disks of their own. An OpenVMS node sees such a disk with `SHOW DEVICE`, initializes it with `INITIALIZE` and mounts it with `MOUNT`; `COPY`, `BACKUP` and `ANALYZE/DISK_STRUCTURE` work on it as on a local disk. Several nodes can mount one disk at the same time with `MOUNT/CLUSTER`.

Nothing is installed on OpenVMS, and no OpenVMS parameter is changed.

1.2 How an OpenVMS Node Sees the Server

The server speaks NISCA, the cluster protocol that PEDRIVER speaks on the LAN (Ethernet type 60-07). To an OpenVMS node it is a node of the cluster that has no votes and runs no connection manager - in the way an HSC controller is on a CI. It is therefore never a member of the VMScluster: it cannot take part in a quorum, it cannot hang the cluster, and a node can lose it without a state transition.

The server has a name (SCSNODE) and a system ID (SCSSYSTEMID) of its own. The OpenVMS node shows it in `SHOW CLUSTER` and in `MCR SCACP SHOW CHANNEL`, opens a virtual circuit to it, and connects its disk class driver to the disk server of MSCPHUB, `MSCP$DISK`. The disks appear on the node as `$`*n*`$DK`*cn* (for example `$42$DKA100`), *n* being the allocation class of the server.

1.3 What Is Served

Item	What the server does
Disks	Up to 8 per server (`/UNIT0` to `/UNIT7`). A disk is a Linux file or a block device: a whole disk, a partition, an LVM volume, a loop device, an iSCSI LUN.
Sectors	Devices of 512-octet and of 4096-octet sectors. OpenVMS always works in blocks of 512 octets; the server does the rest.
Read-only disks	A disk can be served read only - for example the CD image of OpenVMS, as an RRD42.
Commands	READ, WRITE, ERASE, COMPARE HOST DATA, ONLINE, AVAILABLE, GET UNIT STATUS and the other MSCP commands an OpenVMS disk class driver sends.
Nodes	Every OpenVMS node of the group on the LAN, at the same time.

1.4 When a WRITE Is Done

OpenVMS never asks a disk to empty a cache of its own: when a disk says that a WRITE is done, OpenVMS takes it that the data is on the medium. The server therefore answers a WRITE only when its data is on the disk of the Linux computer. This is the default and it is safe against a crash or a power failure of the Linux computer. The qualifier `/UNSAFE` turns it off, for tests only.

1.5 What MSCPHUB Does Not Do

- It is not a member of the VMScluster and has no votes.
- It serves disks only: no tapes, and no boot of diskless satellites.
- It cannot yet be a member disk of a shadow set.
- It works over the LAN only. A cluster over IP (IPCI) is not supported.
- It uses one LAN adapter.

MSCPHUB is verified with OpenVMS Alpha Version 8.3. Other versions that cluster over the LAN with PEDRIVER are expected to work.

1.6 Small Computers

MSCPHUB needs little: the C library, liburing, the StarLet package and a kernel of the last five years - no database, no interpreter, no framework. The server is 250 to 300 KB, MHMON about 150 KB, and a running server takes under 3 MB of memory, and its data buffers as the commands need them, at most 16 MB.

So the hub of a home VMScluster can be a small board. The same kit builds and passes all its tests on a NaPi-C board of the NaPi World (https://github.com/napilab/napi-boards): a Rockchip RK3308, four Cortex-A35 cores, half a gigabyte of memory, Armbian. A Raspberry Pi or any other ARM board with a 64-bit Linux should do as well. Put the images on a USB disk or an SSD rather than on the SD card: the server writes every block it is given before it answers.

An emulated Alpha on a desktop computer, a NaPi in the corner serving its disks, and the cluster is complete - for the enthusiast, the way we like it.

Chapter 2. Installation

2.1 Requirements

- A Linux computer with a kernel of the last five years (5.10 or later; Debian 11 or later).
- An Ethernet connection to the LAN of the OpenVMS nodes: the same switch, VLAN or bridge.
- The StarLet package, version 1.6 or later, installed.
- The packages `cmake`, `gcc`, `pkg-config` and `liburing-dev`.
- Optionally the HELP package: with it, `/HELP` displays the whole help library.
- root on Linux.

2.2 Building and Installing

MSCPHUB is built from its kit, `MSCPHUB-`*ident*`.tar.gz`:

```
# apt install cmake gcc pkg-config liburing-dev
# tar xzf MSCPHUB-X01-19.tar.gz
# cd MSCPHUB-X01-19
# cmake -S . -B build
# cmake --build build -j
# ctest --test-dir build
# cmake --install build
```

Install only when `ctest` says `100% tests passed`.

2.3 What Is Installed

With the default prefix, `/opt/mscphub`:

File	What it is
`/opt/mscphub/sbin/mscphub`	The server
`/etc/mscphub/mscphub.conf.example`	A sample parameter file
`/lib/systemd/system/mscphub.service`	The systemd unit; not enabled
`/etc/init.d/mscphub`	Or the script of OpenRC or of System V init; not enabled
`/opt/mscphub/share/man/man8/mscphub.8`	The manual page
`/opt/mscphub/share/help/mscphub.hlb`	The help library (with the HELP package)
`/opt/mscphub/share/doc/mscphub/`	The manuals and README

Up to Version X01-18 the parameter file was in `/etc/opt/mscphub`. It is not moved for you; the installation warns while it is there, and the server started by the service does not read it. Move it once:

```
# mv /etc/opt/mscphub/mscphub.conf /etc/mscphub/
```

The build looks at the computer and installs one kind of service:

The computer runs	What is installed
systemd (Debian, Ubuntu, Armbian, Fedora)	`mscphub.service`, `mscphub-t4.service` in `/lib/systemd/system`
OpenRC (Alpine, Gentoo, Artix)	the scripts `mscphub`, `mscphub-t4` in `/etc/init.d`
System V init (Devuan, Slackware, busybox)	the scripts `mscphub`, `mscphub-t4` in `/etc/init.d`

To choose yourself, give `-DMH_SYSTEMD=ON` or `OFF`, and `-DMH_INIT=OPENRC`, `SYSV` or `NONE`, to the first `cmake`. None of them is enabled. Chapter 4 gives the commands of systemd; Section 4.11 gives those of OpenRC and of System V init.

2.4 Search Paths

The server and its manual page are not on the search paths of the shell. Add them, for example in `/root/.profile`:

```
export PATH=$PATH:/opt/mscphub/sbin
export MANPATH=$MANPATH:/opt/mscphub/share/man
```

2.5 Installing in Another Place

Give the prefix when the build is configured:

```
# cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/usr/local
```

The parameter file is then in `/usr/local/etc/mscphub`; `-DMH_CONFDIR=`*directory* puts it elsewhere.

2.6 Verifying the Installation

```
# mscphub /HELP
```

displays the description of the server. No OpenVMS node is needed for it.

2.7 Removal

From the build directory of the installation:

```
# cmake --build build --target uninstall
```

This removes what that installation put in place, and nothing else. The parameter file `mscphub.conf` is yours and is left.

Chapter 3. Configuration

3.1 The Facts of the Cluster

Write down four things on one OpenVMS node of the cluster.

The group number:

```
$ MCR SYSMAN CONFIGURATION SHOW CLUSTER_AUTHORIZATION
```

The password hash. It is in `SYS$SYSTEM:CLUSTER_AUTHORIZE.DAT`:

```
$ DUMP SYS$SYSTEM:CLUSTER_AUTHORIZE.DAT
```

Take the quadword at offset 8 and write its octets from the lowest one. The longwords `00000000 8EF54F24` there give `/AUTHORIZE=5A3C9E1700000000`.

The names and the system IDs in use:

```
$ SHOW CLUSTER
$ MCR SYSGEN SHOW SCSSYSTEMID
```

Choose for the server a name (`/NODE`, up to 6 characters) and a system ID (`/SYSTEMID`, 1 to 65535) that no node of the cluster has. The server checks this itself when it starts, and stops if it finds a clash.

An allocation class for the disks, 1 to 255, that the cluster does not use for disks of the same names. With `/ALLOCLASS=42` the disk `DKA100` is `$42$DKA100:` on every OpenVMS node.

3.2 Preparing the Disks

3.2.1 A File

A new empty file of 2 GB:

```
# mkdir -p /srv/mscphub
# truncate -s 2G /srv/mscphub/dka100.img
```

Such a file is *sparse*: it takes no space yet, and the first write to every part of it is slower. For a disk that must be fast, write the file full once instead:

```
# dd if=/dev/zero of=/srv/mscphub/dka100.img bs=1M count=2048
```

The size of the file when the server starts, in blocks of 512 octets, is the size of the disk.

Put the files on ext4 or XFS. The server reads and writes them with direct I/O, as it does a block device. On a file system without direct I/O (tmpfs of an old kernel, some network file systems) it uses the page cache and says `NODIRECT`; the disk works, but every write costs more.

3.2.2 A Block Device

A whole disk, a partition, an LVM volume, a loop device or an iSCSI LUN. It must not be mounted or used by Linux: the server takes it for itself and refuses one that is in use (`DEVBUSY`). A partition or an LVM volume is a little faster than a file.

3.2.3 A Disk of 4096-Octet Sectors

Such a disk works. A partition on it must start on a physical sector, or every write is slow (the server says `MISALIGN`); `parted` and `fdisk` start partitions on 1 MB by default. Section 4.3 says how to initialize such a disk on OpenVMS.

3.2.4 A CD Image

The CD of OpenVMS, an ISO file, is served read only, as an RRD42:

```
/UNIT2=DKA300,/srv/mscphub/AXPVMS083.iso,RO
```

Caution

> Never give one file or device to two servers, and never mount it on Linux while it is served.

3.3 The Parameter File

```
# cp /etc/mscphub/mscphub.conf.example /etc/mscphub/mscphub.conf
# vi /etc/mscphub/mscphub.conf
```

The file holds one qualifier on a line, as on the command line. A line that begins with `!` is a comment. A complete example:

```
! MSCPHUB of our cluster
/INTERFACE=eth1
/NODE=MHUB
/SYSTEMID=65200
/GROUP=4242
/AUTHORIZE=5A3C9E1700000000
/ALLOCLASS=42
/UNIT0=DKA100,/srv/mscphub/dka100.img
/UNIT1=DKA200,/dev/vg0/vmsdata
/UNIT2=DKA300,/srv/mscphub/AXPVMS083.iso,RO
```

Every qualifier is described in Chapter 5.

Note

A qualifier the server does not know is passed over with a warning (`Skip unrecognized option`), and the server runs without it. Read the log after every change of the file.

3.4 Checking the Configuration

Start the server by hand with `/PASSIVE`. It only listens and sends nothing, so it is safe on any LAN:

```
# mscphub /CONFIG=/etc/mscphub/mscphub.conf /PASSIVE
```

After the 4 seconds of `/LISTEN` there is a `NEWNODE` for every OpenVMS node. Press `Ctrl/C`. No `NEWNODE`: see Chapter 7.

Chapter 4. Operation

4.1 Starting the Server

```
# systemctl enable --now mscphub
# journalctl -u mscphub -f
```

The unit is not started at boot until you enable it, and it does not start without `/etc/mscphub/mscphub.conf`. This is on purpose: a server that starts by itself talks to the cluster on its LAN at once.

4.2 What the Log Shows at Start

In this order:

```
%MSCPHUB-I-UNITCFG, unit $42$DKA100 is /srv/mscphub/dka100.img, 4194304 blocks, a file, sector 512, RZ29, S
%MSCPHUB-I-STARTUP, MSCPHUB X01-16, node MHUB, system ID 65200, group 4242, on eth1 (8E-
4E-65-DB-5C-B3)
%MSCPHUB-I-LISTEN, listening 4 seconds before announcing the node
%MSCPHUB-I-NEWNODE, node ALPHA1, system ID 65534, adapter 08-00-2B-E5-40-01, CC V1.6 (104 octets)
%MSCPHUB-I-ANNOUNCE, node MHUB announced on eth1, 1 other node seen in group 4242
%MSCPHUB-I-CHOPEN, channel to node ALPHA1 (65534) is open, CC V1.6, pipe quota 16, buffer 1426, services 000
%MSCPHUB-I-VCOPEN, virtual circuit to node ALPHA1 (65534) is open: VMS V8.3 on ALPH, datagram 576, message 2
%MSCPHUB-I-CONOPEN, connection VMS$DISK_CL_DRVR of ALPHA1 (65534) to MSCP$DISK is open, credit 10/16
```

`CONOPEN` means that the disk class driver of the node has connected: the disks are there.

4.3 Using the Disks on OpenVMS

A few seconds after `CONOPEN`:

```
$ SHOW DEVICE $42$DKA
$ INITIALIZE $42$DKA100: DATA1
$ MOUNT/SYSTEM $42$DKA100: DATA1
$ COPY *.* $42$DKA100:[000000]
```

On a Linux disk of 4096-octet sectors, give the cluster size yourself, 8 or a multiple of 8:

```
$ INITIALIZE/CLUSTER=16 $42$DKA100: DATA1
```

Then the files fill whole sectors and are written without being read first. OpenVMS takes 16 by itself on a disk of more than 50000 blocks, and 1 on a smaller one.

Two or more nodes can use one disk together:

```
$ MOUNT/CLUSTER $42$DKA100: DATA1
```

4.4 Displaying the State of the Server

```
# systemctl kill -s USR1 mscphub
# journalctl -u mscphub -n 40
```

On `SIGUSR1` the server writes to the log, in this order:

Message	What it reports
`UPTIME`	The version, the name, how long the server has run
`NODESTATS`	One for each node: the channel, the virtual circuit, the connections open
`STATS`	The frames received and sent
`TRSTATS`	The transport: packets, retransmissions, duplicates
`ACKSTATS`	How the packets of the nodes were acknowledged
`BLKSTATS`	The data moved
`MSCPSTATS`	Commands in progress and the data buffers in use
`UNITSTATS`	One for each disk: hosts, commands, reads and writes
`BKSTATS`	One for each disk: the work of the Linux disk
`LATSTATS`	One for each disk and for each of READ and WRITE: how long the commands took

The counters run from the start of the server. To measure a test, send `SIGUSR1` before and after it and subtract. Chapter 6 describes each message.

4.5 Stopping the Server

Dismount its disks on OpenVMS first. Then:

```
# systemctl stop mscphub
```

The server says `HOSTSHUT` to every node and `BYE` to the group; the OpenVMS nodes close their circuits to it at once. Last, it writes its state and its counters to the log, as on `SIGUSR1`.

A disk still mounted when the server stops waits for it - the node tries again - and the work goes on when the server comes back.

4.6 Altering the Service

To pin the server to a CPU, or to change its start line, use a drop-in, not the unit file:

```
# systemctl edit mscphub
[Service]
CPUAffinity=3
```

4.7 Backing Up the Disks

Back up an image file as you back up any disk: with the server stopped, or with the disk dismounted on all OpenVMS nodes. A copy of a file that a node is writing is not a consistent disk.

4.8 Monitoring the Server

MHMON shows what the server does, as the MONITOR utility of OpenVMS does. The server makes a copy of its state and counters every 100 ms in shared memory; MHMON reads it and never writes to it, so it can run at any time and as often as you like. Run it as root on the Linux computer of the server:

```
# mhmon
```

The class DISK is the default. For every disk it shows the name the nodes give it, the label of the volume, the nodes that have it online - that is, mounted or being mounted - and the item, with the value of the last interval (CUR) and the average, the least and the greatest since MHMON started:

```
                          MSCPHUB Monitor Utility
                             DISK I/O STATISTICS
                               on server MHUB
                            22-OCT-2026 14:32:29

I/O Operation Rate                                  CUR      AVE      MIN      MAX

$42$DKA100   MHUBTEST     ALPHA1,ALPHA2         377.33   396.56   377.33   416.66
$42$DKA200   ALPHASYS     ALPHA1                  0.00     0.00     0.00     0.00
$42$DKA300   ALPHA083     -                       0.00     0.00     0.00     0.00
$42$DKA400   MHUB4K       ALPHA1,ALPHA2           0.00     0.00     0.00     0.00
```

`/ITEM` chooses what is shown: the operation rate, the throughput, the length of the queue, or the response time; `/ITEM=ALL` shows them in turn. The class NODES shows the nodes:

```
Node    System ID  Channel  Circuit  Opens  Connections  Disks  Commands Rate

ALPHA1      65534  OPEN     OPEN         1            1      3          0.00
ALPHA2      65533  OPEN     OPEN         1            1      2        296.50
```

and the class SERVER the frames, the transport and the data. `mhmon ALL` shows the three in turn. Press `Ctrl/C` to stop. Chapter 5 describes every qualifier.

4.9 Recording the Counters in T4 Files

T4 is the timeline collector of OpenVMS; TLViz and other tools draw its files. MHMON writes the counters of the server in the same T4 style CSV, so that they can be drawn with the same tools, beside the T4 data of the OpenVMS nodes.

To record every minute, a file a day, enable the collector that is installed with the server:

```
# systemctl enable --now mscphub-t4
```

The files are in `/var/opt/mscphub/t4`, named as the files of T4: `T4_MHUB_22OCT2026_0000_2400.CSV`. The file of today ends in `_RUN` while it is being written. To record by hand:

```
# mhmon ALL /NODISPLAY /RECORD=/tmp/test.csv /INTERVAL=10 /ENDING=18:00
```

A file begins with four rows - the node, the date, the time of the start, the names of the columns - then has a row a sample: the time of the end of the sample, then the values. The columns are the rows of the class SERVER, four items of every disk, the commands of every node.

4.10 Reading the Counters over Modbus/TCP

A SCADA, a PLC panel or any Modbus client can read the counters of the server. Give the server a port:

```
/MODBUS=502
```

The server then answers the Modbus functions 03 and 04 (read registers) and 2B/0E (read the identification of the device). The counters come from the same copy MHMON reads, served by a thread of their own: a client never holds up the disks. Nothing can be written. Appendix D gives the map of the registers.

4.11 Running Without systemd

On a computer with OpenRC or System V init, the installation puts the scripts `mscphub` and `mscphub-t4` in `/etc/init.d`. The messages of the server go to `/var/log/mscphub.log`, those of the collector of T4 to `/var/log/mscphub-t4.log`. The commands of this chapter become:

To	systemd	OpenRC	System V init
Start at boot	`systemctl enable mscphub`	`rc-update add mscphub default`	`update-rc.d mscphub defaults`
Start	`systemctl start mscphub`	`rc-service mscphub start`	`/etc/init.d/mscphub start`
Display the state	`systemctl kill -s USR1 mscphub`	`rc-service mscphub report`	`/etc/init.d/mscphub report`
Read the log	`journalctl -u mscphub`	`less /var/log/mscphub.log`	`less /var/log/mscphub.log`
Stop	`systemctl stop mscphub`	`rc-service mscphub stop`	`/etc/init.d/mscphub stop`
Record T4 files	`systemctl enable --now mscphub-t4`	`rc-update add mscphub-t4 default`	`update-rc.d mscphub-t4 defaults`

`update-rc.d` is the command of Devuan; on Slackware, call the script from `/etc/rc.d/rc.local`.

To alter the service, do not edit the script - the next installation overwrites it. Set its variables in a file of your own:

OpenRC: `/etc/conf.d/mscphub`	System V init: `/etc/default/mscphub`	Meaning
`mscphub_config`	`MSCPHUB_CONFIG`	The parameter file
`mscphub_log`	`MSCPHUB_LOG`	The log
`mscphub_cpus`	`MSCPHUB_CPUS`	The CPUs the server runs on, as `taskset -c` takes them

For example, to pin the server to CPU 3:

```
# echo 'MSCPHUB_CPUS=3' > /etc/default/mscphub
```

The busybox of Alpine has no `taskset`: `apk add util-linux-misc` first.

The collector of T4 takes `mhmon_dir`, `mhmon_interval`, `mhmon_log` and `mhmon_node` from `/etc/conf.d/mscphub-t4`, or `MHMON_DIR`, `MHMON_INTERVAL`, `MHMON_LOG` and `MHMON_NODE` from `/etc/default/mscphub-t4`.

Under OpenRC, `supervise-daemon` starts the server again 5 seconds after it fails, as systemd does. Under System V init, nothing does: the script says so when the server stops at once, and the log says why.

Chapter 5. Command Dictionary

This chapter describes the `mscphub` and `mhmon` commands and each of their qualifiers.

A qualifier is given on the command line or, one on a line, in the file of `/CONFIG`. `--`*NAME*`=`*value* may be written for `/`*NAME*`=`*value*. A qualifier the server does not know is passed over with a warning.

MSCPHUB

Starts the MSCP disk server.

Format

```
mscphub /CONFIG=file-spec
mscphub /INTERFACE=name /NODE=name /SYSTEMID=n /GROUP=n /AUTHORIZE=hex [/qualifier...]
mscphub /HELP["topic..."]
```

Parameters

None.

Description

The server announces itself on the LAN of `/INTERFACE` after `/LISTEN` seconds, and checks that no node already uses its name or its system ID. It opens a channel and a virtual circuit to every OpenVMS node of the group, and serves the disks of `/UNIT0` to `/UNIT7` to every node that connects to its disk server. It runs until it receives `SIGINT` or `SIGTERM`.

The server needs `CAP_NET_RAW`: run it as root. The qualifiers `/INTERFACE`, `/NODE`, `/SYSTEMID`, `/GROUP` and `/AUTHORIZE` are required.

The signals the server takes:

Signal	Action
`SIGINT`, `SIGTERM`	Stop the server (message `SHUTDOWN`)
`SIGUSR1`	Write the state and the counters to the log (Section 4.4)

The image returns 0 to the shell when it stops on a signal, and 1 when it stops on a condition of severity F.

Qualifiers

/ACK={`HOLD` | `FLUSH`} Specifies when the server acknowledges the packets of a node in a packet of its own. Most of the time the acknowledgement travels in a packet of the server - the answer to a command, the data of a READ. When the server has nothing to send, it sends the acknowledgement alone.

HOLD sends the acknowledgement alone only when the node asks for it at once, after 4 packets, or after 20 ms at most; otherwise it waits for the answer of the server. FLUSH sends it as soon as the server has taken all the frames that came in.

The default is /ACK=HOLD. HOLD sends far fewer packets; the messages ACKSTATS and LATSTATS compare the two.

/ALLOCLASS=*n* Specifies the allocation class of the server, 0 to 255. With /ALLOCLASS=42 the OpenVMS nodes name the disk DKA100 of the server `$42$DKA100`; with 0 they name it *node*`$DKA100`, after the SCSNODE of the server. The default is 0.

Do not change it while OpenVMS nodes use the disks, and do not give a class that the cluster uses for disks of the same names.

/AUTHORIZE=*hex* Specifies the cluster password, as the 8-octet hash OpenVMS keeps in `SYS$SYSTEM:CLUSTER_AUTHORIZE.DAT`: 16 hexadecimal digits, the octets from the lowest one. Required. Section 3.1 says how to read it.

/CONFIG=*file-spec* Reads the qualifiers from the file named, one on a line. A line that begins with `!` is a comment. The qualifiers of the command line are taken too.

/CREDITS=*n* Specifies how many commands each OpenVMS node may have in progress on the disk server at a time, 2 to 64. The value is given to a node when it connects. The default is 16.

/GROUP=*n* Specifies the cluster group number, 1 to 65279. Required.

/HELP[=*"topic..."*] Displays the help library of the server: the whole description, or the topic named, and stops.

/INTERFACE=*name* Specifies the Linux interface the OpenVMS nodes are on: an Ethernet adapter, a bridge, a VLAN, a tap device in a bridge. Required. The MAC address of the interface is not changed.

/LISTEN=*seconds* Specifies how long the server listens before it sends anything, 1 to 60 seconds. In that time it learns the nodes of the LAN from their HELLO messages; a node that uses the name or the system ID of the server stops it. The default is 4.

/LOGFILE=*file-spec* Writes the messages to the file named. By default they go to the standard error, which systemd keeps in its journal; the scripts of OpenRC and System V init send it to `/var/log/mscphub.log`.

/MODBUS=[*address*:]*port* Serves the counters of the server over Modbus/TCP on the port named, and on the address named or on every address of the computer. The registers are read with the functions 03 and 04, the identification of the device with 2B/0E; nothing can be written. A thread of its own answers, out of the copy of the counters the server makes every 100 ms. Without /MODBUS nothing is served. Appendix D gives the map of the registers.

/NODE=*name* Specifies the SCSNODE of the server, 1 to 6 of the characters A-Z, 0-9, `$` and _. Required. It must not be the name of any other node of the group.

/PASSIVE The server only listens and reports the nodes it sees. It opens no channel and sends nothing at all. Safe on any LAN, for example to look at a production cluster.

/SYNC={`DSYNC` | `GROUP` | `NONE`} Specifies how a WRITE is made durable before it is answered.

DSYNC writes every WRITE to the medium by itself. GROUP makes the writes that end while the disk is being flushed wait for the next flush, and answers them together: fewer flushes when many nodes write at once, and as safe as DSYNC. NONE is the same as /UNSAFE.

The default is /SYNC=DSYNC.

/SYSTEMID=*n* Specifies the SCSSYSTEMID of the server, 1 to 65535. Required. It must differ from that of every other node of the group; on the LAN the nodes are told apart by the low 16 bits of the SCSSYSTEMID only.

/TIMEZONE={`LOCAL` | `UTC` | [+|-]*hh*[:*mm*]} Specifies the zone of the time the server tells the nodes when it opens a circuit. `SHOW CLUSTER` and SCACP show it as the time the server came up. LOCAL is the zone of the Linux computer, summer time and all. The default is LOCAL. Only the operator sees this time; the protocol works with any zone.

/TRACE Writes every frame received and every step of every channel to the log. Use it for a short time only.

/UNIT0=*name*,*path*[,`RO`][,`MEDIA=`*medium*][,`SECTOR=`*octets*] ... /UNIT7=... Specifies a disk. *name* is `DK`*cn*: a controller letter A to Z and a number 0 to 9999, for example DKA100. *path* is a Linux file or a block device (Section 3.2).

The options, in any order:

`RO` - read only. The node mounts the disk write-locked.

`MEDIA=`*medium* - what the node is told the disk is: two or three letters and a number. The default is RZ29, and RRD42 with RO.

`SECTOR=`*octets* - takes a file as a disk of 512, 1024, 2048 or 4096-octet sectors. For tests of 4K disks only; a block device tells its sector itself.

The server takes a file for itself with a lock, and opens a block device for exclusive use: a second server cannot serve it, and a device mounted on Linux is refused (`DEVBUSY`).

/UNSAFE Answers a WRITE as soon as Linux has taken the data, before it is on the disk. Faster, but a crash or a power failure of the Linux computer can lose data an OpenVMS node believes is written. For tests only. The server says `UNSAFE` when it starts.

Examples

```
# mscphub /CONFIG=/etc/mscphub/mscphub.conf /PASSIVE
```

Starts the server with its parameter file, to listen only. The server reports every node it sees and sends nothing.

```
# mscphub /INTERFACE=eth1 /NODE=MHUB /SYSTEMID=65200 /GROUP=4242 /AUTHORIZE=5A3C9E1700000000 -

  /ALLOCLASS=42 /UNIT0=DKA100,/srv/mscphub/dka100.img
```

Starts the server by hand with one disk; the nodes see it as `$42$DKA100`. (The line is broken here for print; type it as one.)

```
# mscphub "/HELP=Messages IDCLASH"
```

Displays the description of the message IDCLASH.

MHMON

Shows the state and the counters of the running server, and records them in T4 files.

Format

```
mhmon [DISK | NODES | SERVER | ALL] [/qualifier...]
mhmon /HELP["topic..."]
```

Parameter

class The class to show; any unique leading part of its name will do:

DISK - every disk: the label of the volume, the nodes that have it online, the item of /ITEM. The default.

NODES - every node: its system ID, its channel and virtual circuit, the times the channel has opened, its connections, the disks online to it, the MSCP commands it sends a second.

SERVER - frames, kilobytes, retransmissions, duplicate and out of order packets, data sent and taken, commands in progress, data buffers.

ALL - the three in turn, one an interval.

Description

MHMON reads the copy of the state and the counters that the server makes every 100 ms in shared memory, `/dev/shm/mscphub.`*node*, and never writes to it. Every /INTERVAL seconds it shows, for every item, the value of the last interval (CUR) and the average, the least and the greatest since it started (AVE, MIN, MAX). Rates are taken over the times the server has stamped on its copies.

On a terminal the screen is drawn anew every interval; otherwise the screens follow one another, so the output can go to a file. A server that restarts starts the statistics afresh; one that stops is said on the screen, and MHMON waits for it.

The label of a volume is read from the home block of the disk, every minute and whenever the nodes that have the disk online change. The file or the device is opened read only.

Run MHMON as root, on the computer of the server.

Qualifiers

/ENDING=*hh*:*mm*[:*ss*] Specifies when MHMON stops: today, or tomorrow when the time is past. Without it MHMON runs until `Ctrl/C` or SIGTERM.

/INTERVAL=*seconds* Specifies the seconds between two samples, 1 to 3600. The samples fall on the boundaries of the interval of the clock: with /INTERVAL=60, on the minute. The default is 3.

/ITEM={`OPERATION_RATE` | `THROUGHPUT` | `QUEUE_LENGTH` | `RESPONSE_TIME` | `ALL`} Specifies what the class DISK shows: commands a second; kilobytes a second; the average number of commands running and waiting, taken every 20 ms; the average time from a command to its answer, in milliseconds. ALL shows them in turn. The default is /ITEM=OPERATION_RATE.

/NODE=*name* Specifies the SCSNODE of the server to watch, when several servers run on the computer. Without it MHMON takes the only one there is.

/NODISPLAY Nothing is shown; MHMON only records. /RECORD is required with it.

/RECORD=*file-spec* | *directory* Writes the values of every interval to a file of the T4 style (Section 4.9). The file is written row by row, as the samples are taken. A directory gets a file a day, `T4_`*node_date_start_end*`.CSV`; the file being written ends in `_RUN`. A recording waits for a server that does not run, and leaves the fields of such samples empty.

Examples

```
# mhmon NODES
```

Shows the nodes of the server every 3 seconds.

```
# mhmon DISK /ITEM=ALL /INTERVAL=5
```

Shows the four items of the disks in turn, every 5 seconds.

```
# mhmon ALL /NODISPLAY /RECORD=/var/opt/mscphub/t4 /INTERVAL=60
```

Records every minute, a file a day, as the unit `mscphub-t4.service` does.

Chapter 6. Messages

6.1 Message Format

The server reports every condition in the OpenVMS form:

```
21-10-2026  14:04:20.462  3127338  [MSCPHUB\main:503]  %MSCPHUB-I-STARTUP,  MSCPHUB  X01-16, node MHUB, ...
```

The fields are the date and time, the process ID, the module and routine that found the condition, then the facility name `MSCPHUB`, the severity, the identification and the text of the message.

The severity letters are:

Letter	Severity	Meaning
`S`	Success	The operation completed
`I`	Information	Something worth knowing; nothing to do
`W`	Warning	Something is not as it should be; the server goes on
`E`	Error	An operation failed; the server goes on
`F`	Fatal	The server stops

6.2 Message Reference

The messages are listed in alphabetical order. A word in *italics* stands for a value the server puts in; "(s)" stands for a plural ending.

ACKSTATS, information acknowledgements (/ACK=*mode*): *n* asked for at once; *n* with a packet of ours, *n* alone: *n* asked, *n* for a duplicate, *n* for *n* packets, *n* when drained, *n* on the tick

Explanation: Written on SIGUSR1 after TRSTATS. How many packets of the nodes asked to be acknowledged at once, how many acknowledgements went in a packet of the server, and how many went alone - because the node asked, for a packet that came twice, for a batch of packets (/ACK=HOLD), when all frames were taken (/ACK=FLUSH), or on the 20 ms tick. Fewer alone is better.

User Action: None.

ANNOUNCE, information node *name state* on *interface*, *n* other node(s) seen in group *n*

Explanation: The listen time is over and the server now sends HELLO messages. With /PASSIVE it says "listens only" and sends nothing.

User Action: None. If no other node is seen, see Section 7.2.

AUTHFAIL, warning node *name* (*systemid*, *adapter*) sent a CC message with a wrong cluster password, ignored

Explanation: The node is in the group of the server, but its password hash is not that of /AUTHORIZE. No channel is opened to the node. The message is written once for each node.

User Action: Give /AUTHORIZE the hash of the cluster of this node (Section 3.1), and check /GROUP.

BKSTATS, information unit *name*: sector *n*, *n* read-modify-write(s) needing *n* sector(s), *n* from the cache (in the last 2/16/128/512 written: *n*/*n*/*n*/*n*), *n* wait(s) for a range, *n* group fdatasync(s) for *n* write(s), *n* ZERO(s) (*n* as zeros written)

Explanation: Written on SIGUSR1 for each disk, after its UNITSTATS: the sector of the disk; how many writes were done as read-modify-write (a disk of 4096-octet sectors) and how many sectors they needed first; how many of those came from the cache of the 128 sectors last written; how many were among the last 2, 16, 128 and 512 written; how many writes waited for another one on the same sectors; with /SYNC=GROUP, how many flushes covered how many writes; how many ERASE commands came, and how many of them were done by writing zeros.

User Action: None.

BLKERR, warning *operation* from *name* (*systemid*), XCT_ID *id*: *reason*, dropped

Explanation: A chunk of data came from a node for no command of the server: the command is over, or the node sent the chunk twice. The server drops it. It does no harm to either side.

User Action: None if it is rare. If it is frequent, report it with a log of /TRACE.

BLKSTATS, information block transfers: *n* chunk(s) sent (*n* octets), *n* RET_CNF, *n* REQ_DAT, *n* chunk(s) taken (*n* octets), *n* dropped

Explanation: Written on SIGUSR1 after ACKSTATS: the chunks of data sent to the nodes (READ) and their octets, the confirmations the nodes sent for them, the requests for data the server sent (WRITE), the chunks taken in and their octets, the chunks dropped.

User Action: None.

CHCLOSE, warning channel to node *name* (*systemid*) is closed: *reason*

Explanation: The channel to a node is closed. The reason is one of:

```
BYE received, the node is going down     the node shuts down or crashes
restarted by the node                    the node started a new channel
no answer to the handshake in 5 seconds
nothing heard from the node for 8 seconds
the node has changed its name or system ID
the server is stopping
```

A node that is still up opens the channel again in a few seconds.

User Action: None if the channel opens again. If it does not, or if it closes again and again, see Section 7.3.

CHOPEN, information channel to node *name* (*systemid*) is open, CC V1.*n*, pipe quota *n*, buffer *n*, services *xxxx*/*xxxx*

Explanation: The handshake with the node is over. The message shows the version of the node, its pipe quota, its buffer size and its services (offered/required).

User Action: None.

CONCLOSE, information connection *name* of *node* (*systemid*) is closed: *reason*

Explanation: A connection to the disk server is closed: the node disconnected, its circuit closed, or the server ended it.

User Action: None; the node connects again. If the reason is “an end message was held back”, see MSCPERR.

CONOPEN, information connection *name* of *node* (*systemid*) to *server* is open, credit *n*/*n*

Explanation: An OpenVMS node has connected to the disk server MSCP$DISK: its disk class driver, VMS$DISK_CL_DRVR. The message shows the credits of both sides. The disks of the server now appear on the node.

User Action: None.

DEVBUSY, fatal the device *device* is in use: mounted on this host, or another server has it

Explanation: The server cannot open a block device of /UNIT*n* for itself: it is mounted on the Linux computer, it is in use by LVM or RAID, or another server has it. The server stops.

User Action: Dismount the device, or give another one. Never serve a device the Linux computer uses.

IDCLASH, fatal node *name* (*adapter*) uses the system ID low word *n* of this server

Explanation: Another node uses the same low 16 bits of the SCSSYSTEMID as the server. Two such nodes cannot be told apart on the LAN. The server stops and sends nothing.

User Action: Give the server another /SYSTEMID. On OpenVMS, `SHOW CLUSTER` or `MCR SYSGEN SHOW SCSSYSTEMID` shows the IDs in use.

IOERR, error *operation* of unit *name*, LBN *n*, *n* octets, failed: errno=*n* (*text*)

Explanation: The Linux file or device of a disk could not be read or written. The message shows the command, the disk, the block, the size and the error of Linux. The OpenVMS node gets a drive error.

User Action: Check the file system and the disk of the Linux computer. “No space left on device” means that the file is sparse and its file system is full.

IOSTUCK, warning *operation* of unit *name*, LBN *n*, is in the backend for *n* seconds: cancelled

Explanation: A command has been in the Linux disk for 15 seconds. The server cancels it; when the cancel takes, the node gets a drive error. When it does not - Linux is still writing - the disk waits for it.

User Action: Check the Linux disk under the unit: a dead disk, a lost iSCSI or NFS server. Look at `dmesg`.

IVPARAM, fatal illegal value *value* of /*name*: *what is expected*

Explanation: A qualifier has a wrong value. The message says what is expected. The server stops.

User Action: Correct the value.

LATSTATS, information unit *name*, *operation*: *n* command(s), average *n* us (backend *n*, transfers *n*, the rest *n*), longest *n* us; under 0.1/0.3/1/3/10 ms and longer: *n*/*n*/*n*/*n*/*n*

Explanation: Written on SIGUSR1 for each disk and for each of READ and WRITE, after its BKSTATS, when the disk has had such commands. The average time from a command to its answer, in microseconds, and how it was spent: the Linux disk (backend), the data on the LAN (transfers), and the rest - waiting for the disk, the server itself. Then the longest time, and how many commands took under 0.1, 0.3, 1, 3 and 10 ms and longer.

User Action: None.

LISTEN, information listening *n* second(s) before announcing the node

Explanation: The server listens for /LISTEN seconds before it sends anything.

User Action: None.

MANYNODES, warning more than *n* nodes on the LAN, node *name* is not remembered

Explanation: There are more nodes on the LAN than the server remembers (64). A node beyond that is not checked for a clash.

User Action: Report it.

MANYSERVERS, error *n* MSCPHUB servers run on this host: give /NODE

Explanation: MHMON has found the statistics of several servers on the computer and does not know which one to show.

User Action: Give MHMON the SCSNODE of the server: /NODE=*name*.

MBUSERR, warning the counters are not served over Modbus/TCP: *call* failed, errno=*n* (*text*)

Explanation: /MODBUS is given, but the server cannot serve the counters there: most often the port is in use by another program (`bind`, errno=98) or the address is not of this computer (`bind`, errno=99). The server serves its disks without it.

User Action: Give another port or address in /MODBUS, or stop the program that has the port.

MISALIGN, warning *device* starts *n* octets into a physical sector of *n*: every write is a read-modify-write of the disk

Explanation: A block device of /UNIT*n* - most often a partition - does not start on a physical sector of the Linux disk. No write of a node then fills whole sectors, and every one is a read-modify-write. The server still serves the device.

User Action: For a new disk, make the partition start on a multiple of 1 MB (`parted` and `fdisk` do so by default) and copy the data over.

MODBUS, information the counters are served over Modbus/TCP on *address*, port *n*

Explanation: The server answers Modbus/TCP requests for its counters (/MODBUS, Section 4.10).

User Action: None.

MSCPERR, error end message of *operation* to *connection* of *node* (*systemid*) held back: *reason*

Explanation: The server was about to send an answer that would crash the OpenVMS node. It did not send it, and ended the connection. This is an error of the server.

User Action: Report it, with a log of /TRACE.

MSCPSTATS, information MSCP server: *n* command(s) in progress, *n* octets of data buffers in use

Explanation: Written on SIGUSR1 after BLKSTATS. When no node uses the disks, both numbers are 0.

User Action: None. A number that stays above 0 with no node connected is a fault of the server: report it with the log.

NAMECLASH, fatal node *name* (*adapter*, system ID *n*) uses the SCSNODE of this server

Explanation: Another node has the same SCSNODE as the server. The server stops and sends nothing.

User Action: Give the server another /NODE.

NEWNODE, information node *name*, system ID *n*, adapter *adapter*, CC V1.*n* (*n* octets)

Explanation: The server sees a node for the first time: its name, its system ID, its adapter and its version.

User Action: None.

NODESTATS, information node *name* (*systemid*): channel *state*, open *n* time(s); virtual circuit *state*, open *n* time(s); *n* connection(s) open

Explanation: Written on SIGUSR1 for each node the server knows, after UPTIME. OPEN twice and one connection is a node that uses the disks. A count of opens that grows means that the node comes and goes.

User Action: None. If the counts grow, see CHCLOSE and VCCLOSE in the log.

NODIRECT, information *path*: no direct I/O (*reason*), served through the page cache

Explanation: The file system of a file of /UNIT*n* does not allow direct I/O, or the size of the file is not whole blocks of 512 octets. The server serves the file through the page cache: it works, but every write costs more.

User Action: None; for speed, put the image on ext4 or XFS, or serve a partition or an LVM volume.

NOPARAM, fatal the parameter /*name* is mandatory

Explanation: A required qualifier is missing. The server stops.

User Action: Add it.

NOSERVER, error no MSCPHUB server runs on this host*text*

Explanation: MHMON has found no statistics of a server, or the server does not update them - MHMON watches them for a quarter of a second when it starts. No server runs on this computer, or not the one /NODE names, or it hangs, or MHMON is not run as root. With /RECORD, MHMON waits for the server.

User Action: Start the server (`systemctl start mscphub`); check /NODE.

NOSTAT, warning no statistics for MHMON: *call* failed, errno=*n* (*text*)

Explanation: The server cannot make its statistics block in shared memory (`/dev/shm`). It serves its disks, but MHMON and /MODBUS see nothing.

User Action: Check that `/dev/shm` is there and is not full (`df /dev/shm`).

NOTOPIC, error sorry, no documentation on *topic*

Explanation: /HELP names a topic the help library does not have.

User Action: Give `mscphub /HELP` to see the topics.

RECORD, information recording every *n* second(s) to *file*

Explanation: MHMON has begun a file of /RECORD.

User Action: None.

SHUTDOWN, information signal *n* received, the server is stopping

Explanation: The server got SIGINT or SIGTERM. It sends HOSTSHUT on every open virtual circuit and, if it has announced itself, BYE to the group: the OpenVMS nodes close their circuits and channels to it at once.

User Action: None.

STARTUP, information MSCPHUB *ident*, node *name*, system ID *n*, group *n*, on *interface* (*address*)

Explanation: The server has started.

User Action: None.

STATS, information rx *n* frame(s) (*n* CC, *n* TR, *n* dropped, *n* own, *n* bad password), tx *n* frame(s), *n* error(s)

Explanation: Written on SIGUSR1 after the NODESTATS: frames received and sent, and send errors. Dropped frames are frames of other groups or for other nodes; that is normal.

User Action: None.

STATVERS, error the statistics of *name* are of version *n*, MHMON reads version *n*

Explanation: The server and MHMON are of different versions, and the layout of the statistics has changed between them.

User Action: Use the MHMON installed with the server; install both from one kit.

SYSERR, fatal *call* failed, errno=*n* (*text*)

Explanation: A Linux system call failed. The server stops. The most frequent ones:

```
socket(AF_PACKET), errno=1      the server is not run as root (CAP_NET_RAW)
ioctl(SIOCGIFINDEX), errno=19   there is no such interface
```

User Action: Correct what the call and errno name.

TRSTATS, information transport: *n* packet(s) sent, *n* retransmitted, *n* received, *n* duplicate(s), *n* out of order, *n* of an old channel; window up to *n*

Explanation: Written on SIGUSR1 after STATS: packets sent, retransmitted, received, duplicates, out of order, packets of an old channel, and the widest the send window has been.

User Action: None. A few retransmissions are normal; many “out of order” mean that frames from the nodes are lost (Section 7.5).

UNITCFG, information unit *name* is *path*, *n* block(s)*attributes*, SLUN *xxxx*

Explanation: At start-up, for every disk: its name on the OpenVMS nodes, its file or device, its size, its kind, sector and medium, and its MSCP unit number.

User Action: None.

UNITSTATS, information unit *name*: ONLINE to *n* host(s), *n* command(s) running, *n* waiting, *n* read(s) (*n* octets), *n* write(s) (*n* octets)

Explanation: Written on SIGUSR1 for each disk, after MSCPSTATS: to how many nodes the disk is online (mounted, or being mounted), how many commands run on it and how many wait for their turn, how many reads and writes it has done.

User Action: None.

UNSAFE, warning /SYNC=NONE: a WRITE is answered before its data is on disk - for tests only

Explanation: /UNSAFE or /SYNC=NONE is given.

User Action: Do not use it with data you need.

UPTIME, information MSCPHUB *ident*, node *name*, up *n* day(s) *hh*:*mm*:*ss*, OpenVMS time of *zone*

Explanation: The first message on SIGUSR1: the version, the name of the server, how long it has run and the zone of /TIMEZONE.

User Action: None.

VCCLOSE, warning virtual circuit to node *name* (*systemid*) is closed: *reason*

Explanation: The virtual circuit to a node is closed. The reason is one of:

`the channel is closed ...`	`see CHCLOSE`
`restarted by the node (START on the open path)`	`the node rebooted`
`HOSTSHUT received, the node is shutting down`	
`no acknowledgement after 30 retransmissions`	`frames are lost`
`a block transfer the server does not take`	
`the server is stopping`	

After a failure the server restarts the channel at once, and the node opens the circuit again in a few seconds.

User Action: None if the circuit opens again. If it closes again and again, see Section 7.5.

VCOPEN, information virtual circuit to node *name* (*systemid*) is open: *software version* on *hardware*, datagram *n*, message *n*

Explanation: The node has answered the server, and `SHOW CLUSTER` on it now shows the server. The message shows what the node told of itself: its software, version and hardware, its largest datagram and message.

User Action: None.

Chapter 7. Troubleshooting

Read the last messages of the log first:

```
# journalctl -u mscphub -n 50
```

`mscphub "/HELP=Messages`*NAME*`"` says what a message means and what to do; so does Chapter 6.

7.1 The Server Stops at Once

The last message is of severity F and says why:

Message	What to do
`F-NOPARAM`, `F-IVPARAM`	Correct the qualifier it names in the parameter file
`F-SYSERR, socket(AF_PACKET), errno=1`	Run the server as root
`F-SYSERR, ioctl(SIOCGIFINDEX), errno=19`	Correct the name of /INTERFACE
`F-IDCLASH`, `F-NAMECLASH`	Give another /SYSTEMID or /NODE
`F-DEVBUSY`	The device is mounted or used on Linux

A qualifier misspelled in the file gives `Skip unrecognized option:` it is not used, and the server runs without it.

7.2 No Node Is Seen

`ANNOUNCE` says 0 other nodes, and no `NEWNODE` comes.

1. Check /GROUP.
2. Check that the interface is up and in the right bridge or VLAN.
3. On OpenVMS, check that PEDRIVER runs: `MCR SCACP SHOW CHANNEL`.
4. Look at the wire:

```
# tcpdump -i eth1 ether proto 0x6007
```

7.3 No Channel Opens, or It Opens and Closes Again

The nodes are seen, but no `CHOPEN` comes:

- Look for `AUTHFAIL`: the hash of /AUTHORIZE is not that of the group.
- Check that /PASSIVE is not given.
- On OpenVMS, `MCR SCACP SHOW CHANNEL` shows the channel to the server and its state.

`CHOPEN` and `CHCLOSE` again and again: read the reason in `CHCLOSE`. “nothing heard from the node” means that frames are lost between the node and the server: check the switch, the bridge or the VLAN.

7.4 The Disks Do Not Appear on the Node

The channel and the circuit are open, but there is no `CONOPEN`.

- Check that the parameter file has /UNIT*n* lines (`UNITCFG` at start-up).
- On OpenVMS, check that the process CONFIGURE runs (`SHOW SYSTEM`): it loads the disk class driver.

If the channel is open but no `VCOPEN` comes, run the server with /TRACE for a short time and look for `PPD START` in the log, and for a warning after it.

7.5 The Circuit Closes, or Writes Are Slow

Read the reason in `VCCLOSE`. Send `SIGUSR1` and look at `TRSTATS`:

- many retransmissions mean that frames to the nodes are lost;
- many “out of order” mean that frames from the nodes are lost.

Writes slow and reads fast is the second case: a node sends a lost frame again only after a second or more. Check the switch, the bridge, the VLAN, the cables.

7.6 A Command Fails on the Node

`MOUNT` or `COPY` fails with an I/O error. Look for:

Message	What it means
`E-IOERR`	The Linux file or disk failed: full, read only, broken
`W-IOSTUCK`	The Linux disk hangs: look at `dmesg`
`E-MSCPERR`	An error of the server: report it
`W-BLKERR`	A stray chunk of data; harmless if rare

7.7 The Disks Are Slow

Message or sign	What to do
`I-NODIRECT`	Move the image to ext4 or XFS
`W-MISALIGN`	Make a new partition on 1 MB
A sparse image	Write it full once (Section 3.2.1)
Many read-modify-writes in `BKSTATS`	INITIALIZE the 4K disk with /CLUSTER=16 (Section 4.3)

`LATSTATS` shows where the time of a command goes: in the Linux disk (backend) or on the LAN (transfers).

7.8 MHMON Shows Nothing

Message or sign	What to do
`E-NOSERVER`	Start the server; run MHMON as root; check /NODE
`E-MANYSERVERS`	Give /NODE
`E-STATVERS`	Use the MHMON of the same kit as the server
`W-NOSTAT` in the log of the server	`/dev/shm` is missing or full
“No MSCPHUB server runs …; waiting for it”	The server has stopped; MHMON goes on when it comes back
`W-MBUSERR` in the log of the server	The port of /MODBUS is in use, or the address is not of this computer

7.9 Collecting Information for a Report

```
# systemctl kill -s USR1 mscphub
# journalctl -u mscphub --since '1 hour ago' > mscphub.txt
```

Add the parameter file, without the line /AUTHORIZE. For a protocol problem, run the server with /TRACE for the minute in which the problem happens, and add that log too.

Appendix A. Files

File	Purpose
`/opt/mscphub/sbin/mscphub`	The server
`/opt/mscphub/sbin/mhmon`	The monitor (Section 4.8)
`/etc/mscphub/mscphub.conf`	The parameter file of the service; yours, never overwritten
`/etc/mscphub/mscphub.conf.example`	A sample parameter file
`/lib/systemd/system/mscphub.service`	The systemd unit
`/opt/mscphub/share/man/man8/mscphub.8`	The manual page
`/lib/systemd/system/mscphub-t4.service`	The collector of T4 files; not enabled
`/etc/init.d/mscphub`, `/etc/init.d/mscphub-t4`	Without systemd: the scripts of OpenRC or of System V init (Section 4.11)
`/etc/conf.d/mscphub`, `/etc/default/mscphub`	Yours: the variables of the script (Section 4.11)
`/var/log/mscphub.log`	Without systemd: the log of the server
`/dev/shm/mscphub.`*node*	The state and the counters of the running server, for MHMON and Modbus
`/var/opt/mscphub/t4/`	The T4 files of the collector
`/opt/mscphub/share/help/mscphub.hlb`	The help library, English
`/opt/mscphub/share/help/ru/mscphub_ru.hlb`	The help library, Russian
`/opt/mscphub/share/doc/mscphub/`	The manuals

Environment Variables

Variable	Meaning
`MSCPHUB_HELPLIB`	The help library /HELP reads, in place of the installed one

Appendix B. Limits

Item	Limit
Disks per server	8 (`/UNIT0` to `/UNIT7`)
Name of a disk	`DK`*c**n*: a letter A to Z, a number 0 to 9999
Size of a disk	The size of the file or the device, in blocks of 512 octets
Sector of a Linux device	512, 1024, 2048 or 4096 octets
Nodes on the LAN remembered	64
Commands in progress per node	/CREDITS, 2 to 64; 16 by default
Data buffers of all commands	16 MB; a command that finds none waits
Packets unacknowledged per circuit	16
Time a disk command may take	15 seconds, then IOSTUCK
Retransmissions before a circuit fails	30
Modbus/TCP clients at a time	8
Copy of the counters for MHMON and Modbus	Every 100 ms
LAN adapters	1

Appendix C. Performance

C.1 The Test Configuration

MSCPHUB is developed and tested against real OpenVMS. The nodes run under AlphaEmu, the emulator of the DEC AlphaServer ES40 made by StarLet Squad: a whole VMScluster of OpenVMS Alpha nodes runs on one Linux computer, next to the server, and every change of MSCPHUB is tried on it before it is released.

Item	Configuration
OpenVMS nodes	Two AlphaServer ES40 under AlphaEmu, 2 CPUs and 1 GB each; OpenVMS Alpha Version 8.3; DE500 LAN adapters
LAN	A Linux bridge on the host
Linux computer	Intel Xeon E5-2697 v4; a consumer NVMe SSD with ext4
MSCPHUB	/SYNC=DSYNC, /ACK=HOLD; the disk is an image file of 200 MB on ext4, written full

C.2 Three Disks Compared

One node writes and reads the same data on three disks in turn: its own IDE disk, a disk of MSCPHUB, and the IDE disk of the other node served to it by the MSCP server of OpenVMS. Seconds, the median of three rounds, MSCPHUB Version X01-16, 22-OCT-2026:

Test	Own IDE disk	MSCPHUB	MSCP server of OpenVMS
`COPY` of a file of 15.7 MB to the disk	0.77	0.99	1.30
`COPY` of that file from the disk to `NLA0:`	4.56	4.78	5.37
`BACKUP/VERIFY` of 1297 small files	27.20	35.47	40.60

MSCPHUB is faster than the MSCP server of OpenVMS in all three tests: by 24 % on the write, 11 % on the read and 13 % on the many small files of BACKUP.

Note

The own IDE disk of an emulated node writes into the memory of the Linux computer, without waiting for its disk, and so does the disk the MSCP server of OpenVMS serves - it is the IDE disk of the other node. MSCPHUB answers a WRITE only when the data is on the medium (Section 1.4).

C.3 Endurance

Two nodes copied files to the disks of MSCPHUB and compared every copy with its original, without a pause:

Run	Version	Result
24 hours, three jobs, a disk of 512-octet and a disk of 4096-octet sectors	X01-09	128 335, 104 056 and 129 025 rounds, 0 errors; the speed the same in the first and the last hour
1 hour, two nodes on one disk of 4096-octet sectors	X01-14	6 541 and 6 695 rounds, 0 errors; `ANALYZE/DISK_STRUCTURE` clean
1 hour, two nodes on one file disk	X01-15	7 895 and 7 856 rounds, 0 errors; `ANALYZE/DISK_STRUCTURE` clean

C.4 Where the Time Goes

The message LATSTATS splits the time of a command. During BACKUP/VERIFY (Version X01-15):

Command	Average	The Linux disk	The LAN and the node	The server itself
READ	550 µs	214 µs	334 µs	1 µs
WRITE	812 µs	551 µs	259 µs	2 µs

The server itself takes one or two microseconds of a command. The rest is the disk of the Linux computer and the node at the other end. The same BACKUP/VERIFY takes 30.7 seconds with /SYNC=NONE and 34.0 to 34.8 on a partition instead of a file: what remains is the price of a durable write on a consumer SSD. An SSD with power-loss protection removes most of it.

C.5 The Server on Another Computer

The server need not run on the computer of the nodes. In this test it ran on a second Linux computer and served a copy of an OpenVMS system disk to the same emulated nodes:

Item	Configuration
Linux computer of the server	Intel Xeon E5-2697 v4 at 2.3 GHz, turbo off; Samsung SSD 970 EVO Plus (NVMe) with ext4
LAN	1 Gb/s Ethernet between the two computers; a VXLAN tunnel joins the bridge of the nodes to a bridge of the server
MSCPHUB	Version X01-19ECO1, /SYNC=DSYNC, /ACK=HOLD; the disk is an image file of 16 GB, a copy of the system disk of a node

Two jobs on one node copied a file of 1.6 MB to the disk and compared every copy with its original, without a pause, for 30 minutes, 22-OCT-2026. The collector of T4 recorded the server every minute:

Measure	Value
Rounds	2 898 and 2 896, 0 errors
I/O operation rate	510 to 550 a second, steady from the first minute to the last
Throughput	15 MB a second: 10 MB read by the node, 5 MB written
Queue length	1.15 to 1.23
Response time	2.2 to 2.3 ms
Commands	753 867 READ (18.9 GB), 191 927 WRITE (9.4 GB)
Transport	14.8 million packets sent, 0 retransmitted, 0 duplicates
`ANALYZE/DISK_STRUCTURE`	No error the load made: it reports only two files marked for delete, which the disk copied already had

LATSTATS of the same run:

Command	Average	The Linux disk	The LAN and the node	The server itself
READ	1 639 µs	235 µs	1 401 µs	1 µs
WRITE	4 650 µs	2 033 µs	2 614 µs	2 µs

Across two computers and a tunnel the server itself still takes one or two microseconds of a command. The way through the LAN and the tunnel costs about a millisecond more than the bridge of C.1, and the durable write on that SSD two milliseconds.

C.6 Losing the Server Under Load

The server of C.5 was then stopped, killed and cut off from the LAN while the node used the disk. Two jobs on one node copied a file of 1.6 MB to the disk and compared every copy with its original for 45 minutes, 22-OCT-2026; in that time the server was lost four times, about seven minutes apart:

Event	The server serves the disk again	What the node shows
`systemctl restart mscphub`	10 seconds later	Nothing: the server said BYE to every node, and the virtual circuit was closed in order
`systemctl stop mscphub`, 90 seconds without a server, `systemctl start mscphub`	10 seconds after the start	Nothing: the I/O of the jobs waited 100 seconds and went on
`kill -9` of the server; systemd starts it again 5 seconds later	19 seconds later	`%PEA0, Excessive packet losses on LAN Path` and `Port has Closed Virtual Circuit`, two entries in the error log
The tunnel down for 120 seconds; the server keeps running	4 seconds after the tunnel is up	The same two messages; the server closed the virtual circuit itself after 8 seconds of silence

Measure	Value
Rounds	3 980 and 3 979, 0 errors; no job failed or stopped
Error count of the disk (`SHOW DEVICE/FULL`)	0
The error log of the node	4 entries of the port PEA0, two for each of the last two events; none of the disk
`ANALYZE/DISK_STRUCTURE`	The same report as in C.5: the four losses left nothing behind

The node keeps every I/O request while the server is away and sends it again when the server is back, as it does for a disk served by another OpenVMS node: the volume goes into mount verification and waits up to MVTIMEOUT seconds (3 600 by default). So the server can be restarted, upgraded or lost for minutes under load, and no data is lost. Stop it in the usual way when you can: the BYE it sends lets the nodes close the virtual circuit at once, without an entry in the error log.

C.7 Reading These Numbers

Under emulation the OpenVMS node is the slowest part of every test, and on real Alpha, Integrity or x86 computers the times are different. What the tables show is the comparison: the same node, the same data, three kinds of disk.

Appendix D. Modbus Register Map

D.1 Conventions

With /MODBUS the server answers the Modbus/TCP functions 03 (Read Holding Registers) and 04 (Read Input Registers) with the same map, and 2B/0E (Read Device Identification). Every Unit Identifier is answered. At most 125 registers are read at a time; a register in the map that holds nothing reads 0; an address past the map is answered with the exception 02, ILLEGAL DATA ADDRESS.

- A register is 16 bits.
- A value of 32 bits takes two registers, a counter of 64 bits four; the high word comes first.
- A text takes two characters a register, the first in the high octet, and is padded with zeros.
- The counters only grow while the server runs, and start from 0 when it starts; a rate is the difference of two readings over the time between them.
- The values are those of the copy the server makes every 100 ms.

The map has a version in register 0. A change of the map raises it.

D.2 The Server, from Register 0

Register	Size	Contents
0	1	Version of the map: 1
1	1	Number of disks
2	1	Number of nodes the server knows
3	1	/SYSTEMID
4	1	/GROUP
5	1	/ALLOCLASS
6	1	/SYNC: 0 DSYNC, 1 GROUP, 2 NONE
7	1	/ACK: 0 FLUSH, 1 HOLD
8	2	Seconds since the server started
10	4	Version of the server, text: X01-19
14	4	/NODE, text
18	8	/INTERFACE, text
100	4	Frames received
104	4	Frames sent
108	4	Octets received
112	4	Octets sent
116	4	Frames dropped (other groups, other nodes)
120	4	Frames with a wrong cluster password
124	4	Send errors
128	4	Packets of the transport sent
132	4	Packets of the transport received
136	4	Packets retransmitted
140	4	Duplicate packets received
144	4	Packets received out of order
148	4	Acknowledgements sent alone
152	4	Chunks of data sent (READ)
156	4	Octets of data sent
160	4	Chunks of data taken (WRITE)
164	4	Octets of data taken
168	4	Octets of data buffers in use now
172	2	Commands in progress now

D.3 A Disk, from Register 1000 + 100 × n

n is the number of the disk, 0 to 7, in the order of /UNIT0 to /UNIT7 that are given.

Offset	Size	Contents
0	8	Name on the nodes, text: 42DKA100
8	4	Medium, text: RZ29
12	1	Bits: 0 - there is a disk, 1 - read only, 2 - a block device (not a file), 3 - direct I/O
13	1	Sector of the Linux device, octets
14	2	Size, blocks of 512 octets
16	1	Number of nodes that have it online
17	1	Commands running now
18	1	Commands waiting for their turn now
20	4	Nodes that have it online: bit *k* is node *k* of D.4
24	4	READ commands
28	4	WRITE commands
32	4	Octets read
36	4	Octets written
40	4	READ commands timed
44	4	Their total time, microseconds
48	4	WRITE commands timed
52	4	Their total time, microseconds
56	4	Sum of the queue (running and waiting), taken every 20 ms
60	4	Number of times it was taken: the average queue is the difference of 56 over that of 60
64	4	Read-modify-writes (a disk of 4096-octet sectors)
68	4	ERASE commands

The average response time of READ over an interval is the difference of offset 44 over that of offset 40.

D.4 A Node, from Register 2000 + 50 × n

n is the number of the node, 0 to 63, in the order the server has seen them.

Offset	Size	Contents
0	4	SCSNODE, text
4	1	SCSSYSTEMID, the low 16 bits
5	1	Channel: 0 closed, 1 CCSTART sent, 2 VERF sent, 3 open, 4 HELLO taken
6	1	Virtual circuit: 0 never started, 1 closed, 2 STACK sent, 3 open
7	1	SCS connections open
8	1	… of them to the disk server
10	2	Times the channel has opened
12	2	Times the virtual circuit has opened
16	4	MSCP commands taken on its connections now open

D.5 Device Identification

The function 2B/0E answers the basic objects:

Object	Name	Value
0	VendorName	StarLet Squad
1	ProductCode	MSCPHUB
2	MajorMinorRevision	X01-19